

Deadline-Aware Task Scheduling for IoT Applications in Collaborative Edge Computing

Seungkyun Lee, SuKyoung Lee^{ID}, and Seung-Seob Lee

Abstract—Collaborative edge computing (CEC) is a promising technology for supporting latency-sensitive Internet of Things (IoT) applications by distributing computation tasks among edge nodes. However, the edge resources are limited compared to the traditional cloud. Therefore, it is important to decide when and where to execute tasks from IoT applications to meet the latency requirements of the applications. In this study, we propose a task scheduling algorithm in the CEC environment, which considers the priority (i.e., deadline) of tasks and network flows with the aim of reducing the deadline miss ratio (DMR) of IoT applications. The simulation results demonstrate that the proposed algorithm can significantly reduce the DMR of other benchmark methods.

Index Terms—IoT, collaborative edge computing, multiple applications, task scheduling, DMR.

I. INTRODUCTION

THE INTERNET of Things (IoT) has brought forth a new generation of applications that are expected to enhance the daily lives of people [1]–[4]. Many of these emerging IoT applications require complex computations and low latency, whereas IoT devices are computationally limited and outsourcing all the computations to a remote cloud causes delays. To resolve this issue, edge computing has been proposed, which offloads the computational tasks received from IoT applications to the edge nodes in close proximity [3].

Meanwhile, collaborative edge computing (CEC) has garnered significant attention because the resources of the edge nodes are quite constrained compared with the traditional cloud on the Internet. It is defined as distributing the computational tasks and data among a distributed network of edge nodes [2]–[5]. For IoT applications in CEC, it is noteworthy that a task executed on an edge node can require input data from different sensing devices located at different geographical locations and are connected to different edge nodes [2], [3], [5]. For instance, healthcare applications require collaboration among the edge nodes in different domains, such as hospitals, pharmacies, insurance providers, and governments [1], [2]. Therefore, while offloading or scheduling tasks from IoT applications to the edge

nodes, when and where to share the input data among the edge nodes will influence the completion time of applications.

Recently, several approaches have been designed for task offloading in the CEC to support latency-sensitive IoT applications [3], [7], [8]. In [3], the authors jointly addressed task offloading, application resource allocation, and task scheduling problems, while satisfying the deadlines of the tasks for IoT applications in CEC. The authors of [7] proposed a priority-aware task offloading and scheduling strategy in a hierarchical fog cloud environment to minimize the waiting time and latency of the tasks while meeting their deadline. In [8], a deep reinforcement learning-based offloading scheme was proposed to jointly maximize the number of computational tasks before their respective deadlines and minimize energy consumption in the CEC. However, these approaches do not address the influence of input data from sensing devices, which are required for completing tasks from IoT applications. Although the authors of [5] studied the data-aware task allocation problem to jointly schedule tasks and data flows in CEC with the objective of minimizing the completion time of the application, they did not focus on meeting the different deadlines of a variety of IoT applications. Therefore, in this letter, we propose a task scheduling algorithm that considers the transfer of distributed input data required by the tasks of multiple applications, while minimizing the deadline miss ratio (DMR) of multiple applications. In contrast to the aforementioned works, the proposed algorithm also considers that there are task dependency requirements for multiple tasks belonging to the same application, noting that an IoT application consists of a set of dependent tasks [5]. Therefore, our algorithm schedules tasks belonging to multiple dynamic acyclic graphs (DAGs). Further, while scheduling based on the deadlines, the proposed algorithm puts the tasks from multiple applications into a common queue and schedules them in parallel to prevent the application with the earliest deadline from being scheduled first always.

Specifically, we first formulate a task scheduling problem that minimizes the DMR for multiple applications in the CEC. As the problem is NP-hard, we propose a heuristic algorithm to solve the formulated task scheduling problem. We then evaluated our task-scheduling algorithm through simulations. The simulation results demonstrate that the proposed algorithm outperforms the benchmark algorithms in terms of the average completion time as well as DMR.

II. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we present the proposed system model. Considering the growing number of IoT users, we assume that multiple applications with differing performance requirements for latency should be run simultaneously in a CEC system [1],

Manuscript received June 7, 2021; accepted July 2, 2021. Date of publication July 8, 2021; date of current version October 7, 2021. This work was supported by the National Research Foundation of Korea (NRF) Grant funded by the Korea Government (MSIT) under Grant 2019R1A2C1086191. The associate editor coordinating the review of this article and approving it for publication was S. Bi. (Corresponding author: SuKyoung Lee.)

Seungkyun Lee and SuKyoung Lee are with the Department of Computer Science, Yonsei University, Seoul 03722, South Korea (e-mail: sklee@yonsei.ac.kr).

Seung-Seob Lee is with the Department of Computer Science, Yale University, New Haven, CT 06511 USA.

Digital Object Identifier 10.1109/LWC.2021.3095496

[3], [5]. Therefore, we consider a CEC system that consists of a set of multiple applications $A = \{a_i\}$, generating tasks with dependencies and a set of edge nodes $V = \{u\}$.

A. Network and Input Data Models

A mesh network of edge nodes forms a CEC system where the edge nodes are connected to each other using a multi-hop path [5], [6]. This communication network is modeled as a connected graph $G = (V, L)$, where $L = \{l_{uv} | u, v \in V\}$ is the set of links connecting the edge nodes. Let p_u and b_{uv} represent the processing power of edge node u and bandwidth capacity of l_{uv} , respectively. In our system model, we assume that the shortest routing path P_{uv} , is given to route the data between the two edge nodes u and v [6]. It is also assumed that no two network flows are allowed to pass through a link at the same time by considering the interference among simultaneous wireless transmissions [5], [6].¹

As IoT applications usually require huge amounts of data generated from IoT devices, the datasets are usually stored at geographically distributed edge nodes. The set of all the data from IoT devices is denoted by $D = \{d_k\}$, where the amount of d_k is denoted by $data_k^{in}$ (in bits). Let $\hat{u}_k$ indicate that the input data source d_k is located at the edge node, $\hat{u}$.

B. Models for Applications and Their Tasks

We assume that each application can be partitioned into multiple tasks that are mutually dependent. Note that tasks belonging to different applications are independent of each other; however, tasks belonging to the same application may have task dependency requirements. We model the tasks belonging to the application a_i as a DAG $G_i = (T_i, E_i)$, where T_i is the set of tasks and E_i the set of directed edges representing dependency between two tasks. More specifically, the directed edge $(t_{ij'}, t_{ij})$ ($t_{ij'}, t_{ij} \in T_i$) represents the precedence relation such that task t_{ij} cannot be scheduled until the task $t_{ij'}$ has finished its processing and has transferred the computation result to t_{ij} . In the given graph G_i , if a task does not have any predecessors (successors), it will be regarded as an entry task (exit task). We assume that a single entry and exit task exists in the graph. We now define each $a_i \in A$ as a pair (δ_i, G_i) , where δ_i is the deadline for the completion time of a_i .

Each task t_{ij} is defined by a four-tuple $(data_j^{res}, c_j, D_j, u_j)$ (we omit the subscript i when the meaning is obvious); where $data_j^{res}$ is the size of the computation result of the task, c_j is the computational load of processing the task, $D_j (\subseteq D)$ is the set of data required for processing the task, and u_j is the edge node where the task is executed.

C. Application Completion Time Model

The completion time of each application a_i , is composed of the computing time of tasks, t_{ij} (denoted by $T_{ij,u}^{com}$), the time to transfer the result from preceding tasks, and the time to transfer the input data. Assuming that each edge can execute

one task at a time [5], [6], the computing time of t_{ij} at the edge node u is given by

$$T_{ij,u}^{com} = \frac{c_j}{p_u}. \quad (1)$$

Note that data transfer occurs when the input data from IoT devices are stored in an edge node that is different from the edge node where the task is executed and when the computation result of one task must be transmitted to its dependent tasks. If the two dependent tasks for a_i , t_{ij} and one of its predecessor tasks (denoted by $\{t_{ij'}\}$) are scheduled to be executed on the edge nodes u and u' , respectively, and $u \neq u'$, then the result from the task $t_{ij'}$ should be transferred from u' to u . It is assumed that the bandwidth for the result is equal to the minimum bandwidth among the links over the routing path, $P_{u'u}$.

$$T_{u'u}^{xfr} = \frac{data_j^{res} \cdot |P_{u'u}|}{\min_{l_{uv} \in P_{u'u}} b_{uv}}, \quad (2)$$

where $|P_{u'u}|$ indicates the number of hops on the path $P_{u'u}$. Similarly, if one of the input data d_k ($d_k \in D_j$) necessary to execute the task t_{ij} at the edge node u should be transferred from the edge node $\hat{u}_k$, we can write the time to transfer the data, d_k as

$$T_{\hat{u}_k u}^{xfr} = \frac{data_k^{in} \cdot |P_{\hat{u}_k u}|}{\min_{l_{uv} \in P_{\hat{u}_k u}} b_{uv}} \quad (3)$$

Let T_u^{fin} be the time when edge node u finishes executing any previously scheduled task. Finally, when task t_{ij} is executed at the edge node u , the start and finish times for task t_{ij} can be expressed as

$$T_{ij,u}^{st} = \max(T_u^{fin}, \max_{\forall \{t_{ij'}\}} (S_{ij',u}^{res} + T_{u'u}^{xfr}), \max_{\forall d_k \in D_j} (S_{ij,\hat{u}_k}^{in} + T_{\hat{u}_k u}^{xfr})) \quad (4)$$

$$T_{ij,u}^{fin} = T_{ij,u}^{st} + T_{ij,u}^{com} \quad (5)$$

where $S_{ij',u}^{res}$ is the start time of the data flow from the preceding task $t_{ij'}$ and $S_{ij,\hat{u}_k}^{in}$ represents the start time of the input data flow to the edge node where task t_{ij} is allocated. Regarding $S_{ij,\hat{u}_k}^{in}$ in Eq. (4), the input data can be transferred any time once the entry task is executed.

D. Problem Formulation

In this section, we present the problem formulation for identifying the edge node u_j , $T_{ij,u}^{st}$, and $T_{ij,u}^{fin}$ such that it minimizes the DMR for all the multiple applications considered. Let X_i indicate whether or not the deadline of the application a_i is satisfied; if $T_{iX}^{fin} > \delta_i$, then X_i is set to 1; otherwise, $X_i = 0$. Then, the DMR is given by

$$DMR = \frac{\sum_{a_i \in A} X_i}{|A|}, \quad (6)$$

where $|\cdot|$ indicates the number of elements in a set. Then, the problem can be formulated to determine the task scheduling decision that minimizes DMR in Eq. (6): however, the problem formulation is NP-hard because the problem of scheduling multiple DAGs to multiple nodes is known to be NP-hard [9]. To solve the problem, we propose a heuristic task scheduling algorithm in the following section.

¹ Different network flows may share the link's bandwidth, but the bandwidth sharing may increase the completion time for some of the flows because they cannot fully use the overall bandwidth, leading to spending more time on transmitting the same amount of data on the link [5].

Algorithm 1 Scheduling for Tasks of Multiple Applications**Input:** RQ, $rank_{ij}$, G_i for $\forall a_i \in A$ **Output:** Task scheduling result

- 1: Sort tasks of RQ in increasing order by δ_i
- 2: **for** $t_{ij} \in \text{RQ}$ **do**
- 3: **while** $RCT > \delta_i$ **do**
- 4: Perform Algorithm 2
- 5: Add the next task in T_i to the head of RQ
- 6: **end while**
- 7: Perform Algorithm 2 for the remaining tasks in RQ
- 8: **end for**

III. PROPOSED ALGORITHM

In this section, we present the details of our proposed scheduling algorithm, which proceeds in two phases: task prioritization and task scheduling. The latter phase determines where and when each task t_{ij} from multiple applications a_i ($1 \leq i \leq |A|$) will be executed.

A. Task Prioritization

We use the upward rank value $rank_{ij}$ of a task $t_{ij} (\in T_i)$, as in the HEFT algorithm [9]. Let $succ(t_{ij})$ denote the set of t_{ij} 's immediate successor tasks; then, we define the rank as

$$rank_{ij} = \bar{T}_{ij}^{com} + \max_{\forall d_k \in D_j} (\bar{T}_{\hat{u}u}^{xfr}) + \max_{t_{ik} \in succ(t_{ij})} (\bar{T}_{u'u}^{xfr} + rank_{ik}), \text{ for } \forall t_{ij} \in T_i, \forall a_i \in A \quad (7)$$

where $\bar{T}_{\hat{u}u}^{xfr}$ is the average time required to transfer the input data from $d_{\hat{u}}$ to t_{ij} . $\bar{T}_{ij}^{com}$ is the average computing time for task t_{ij} on edge nodes, and $\bar{T}_{u'u}^{xfr}$ is the average time required to transfer the result from t_{ik} to t_{ij} . $rank_{ij}$ in Eq. (7) indicates the length of the critical path from task t_{ij} to the exit task. The higher the value, the higher the priority. For each a_i , the tasks are arranged according to the decreasing order of $rank_{ij}$ in the set T_i .

B. Task Scheduling for Multiple Applications

The proposed scheduling algorithm employs a ready queue (RQ) to put the unscheduled task with the maximum $rank_{ij}$ from each of the $|A|$ different applications, a_i ($1 \leq i \leq |A|$). Then, $|A|$ tasks in the RQ are scheduled. Therefore, our algorithm does not schedule the tasks from the application with the earliest deadline first, but the tasks (in the RQ) belong to different applications, while considering the applications' deadlines. The proposed algorithm proceeds as follows (see Algorithms 1 and 2).

- 1) The unscheduled tasks held in the RQ are arranged in increasing order of δ_i .
- 2) For the first task, t_{ij} in the RQ, the remaining completion time (RCT) is calculated as $RCT = \max_{\forall t_{ij'}} T_{ij',u}^{fin} + rank_{ij}$. If the value of RCT is greater than δ_i , then the next task in T_i is added to the head of RQ, and Algorithm 1 is performed until the RCT meets the deadline, δ_i . Otherwise, repeat step 2) to the next task (i.e., the task of another application) in the RQ.

Algorithm 2 Scheduling for a Task, t_{ij} **Input:** Task t_{ij} , G **Output:** $T_{ij,u}^{st}$, $T_{ij,u}^{fin}$, $S_{ij',u}^{res}$, $S_{ij,\hat{u}}^{in}$, Fl_{uv} **Objective:** Find u^* s.t. minimizes $T_{ij,u}^{fin}$

- 1: $Min = \infty$; $(S_0, F_0) = (0, 0) \in Fl_{uv}$
- 2: **for** each $u \in V$ **do**
- 3: **while** $\forall v \in V - \{u\}$ **do**
- 4: $Fl'_{uv} = Fl_{uv}$
- 5: **end while**
- 6: **for** $\forall t_{ij'}$ **do**
- 7: $S_{ij',u}^{res} = \max(T_{ij',u}^{fin}, Fl_{N'u'}^{fin})$
- 8: Add $(S_{ij',u}^{res}, S_{ij',u}^{res} + T_{u'u}^{xfr})$ to $Fl'_{u'u}$
- 9: **end for**
- 10: **for** $\forall d_k \in D_j$ **do**
- 11: **if** $N'_{uv} = 0$ **then**
- 12: $S_{ij,\hat{u}}^{in} = 0$
- 13: Add $(0, T_{\hat{u}u}^{xfr})$ to $Fl'_{\hat{u}u}$
- 14: **else**
- 15: **for** $y = 1; y \leq N'_{uv}; y++$ **do**
- 16: **if** $y \neq N'_{uv}$ **then**
- 17: **if** $(S_y - F_{y-1}) > T_{\hat{u}u}^{xfr}$ **then**
- 18: $S_{ij,\hat{u}}^{in} = F_{y-1}$
- 19: Add $(F_{y-1}, F_{y-1} + T_{\hat{u}u}^{xfr})$ to $Fl'_{\hat{u}u}$
- 20: Break
- 21: **end if**
- 22: **else**
- 23: $S_{ij,\hat{u}}^{in} = F_y$
- 24: Add $(F_y, F_y + T_{\hat{u}u}^{xfr})$ to $Fl'_{\hat{u}u}$
- 25: **end if**
- 26: **end for**
- 27: **end if**
- 28: **end for**
- 29: Compute $T_{ij,u}^{fin}$ using Eq. (5)
- 30: **end for**
- 31: Assign the task t_{ij} to u^* and $Fl_{uv} = Fl'_{u^*v}$

- 3) Perform Algorithm 2 to all the remaining tasks in the RQ to be scheduled.

For the task t_{ij} given from Algorithm 1 (Lines 1-3), which is the first task at the head of RQ, Algorithm 2 determines when and where it will be executed (i.e., $T_{ij,u}^{st}$, $T_{ij,u}^{fin}$, u^*) and the schedule of the network flows (i.e., input data or result) necessary to compute the task (i.e., $S_{ij',u}^{res}$, $S_{ij,\hat{u}}^{in}$). Algorithm 2 prevents any two network flows from passing through a link simultaneously (lines 6-28). Therefore, the set $Fl_{uv} = \{(S_n, F_n) | 0 \leq n \leq N_{uv}\}$ is introduced to record the start and finish times (S_n, F_n) of all the network flows that are transferred on the path P_{uv} , where (S_n, F_n) are arranged in increasing order. In particular, our algorithm considers the input data necessary for computing each task, which is distributed over multiple edge nodes (Algorithms 2 and Lines 10-28).

The computation complexity of the proposed algorithm is $O(|A|^2 \hat{T} |V|^2 \hat{E}^2)$ where $\hat{T}$ is the number of tasks in the worst case and $\hat{E}$ is the maximum number of precedence

between tasks among all the applications considered. The for loop in Algorithm 1 corresponds to the total number of all tasks in multiple applications, which is equivalent to $O(|A|\hat{T})$ in the worst case. Accordingly, Algorithm 1 is completed in $O(|A|\hat{T})$. The complexity of Algorithm 2 depends on the calculation of $T_{ij,u}^{fin}$. The for loop at line 2 has a complexity of $O(V)$. In the worst case, the two for loops at lines 6-9 and lines 10-28 are executed for all possible values of the finish time of data flows $O(\hat{E})$ consisting of the result from precedent tasks and/or input data. Among all the possible values, Algorithm 2 finds the maximum finish time $S_{ij',u}^{res}$ at line 7 by considering all the data flows from the previous tasks for all the applications and all the edge nodes in the routing path, resulting in $O(|A|\hat{E})$ and $O(|V|)$, respectively, in the worst case. The iteration size of the for loop at line 10 is offset by $O(\hat{E})$, and the number of input data for a task is much smaller than $O(\hat{E})$. Therefore, because Algorithm 2 is called by Algorithm 1, the proposed algorithm has a computational complexity of $O(|A|^2\hat{T}|V|^2\hat{E}^2)$ in the worst case.

IV. PERFORMANCE EVALUATION

We performed a simulation to evaluate and compare the performance of the proposed algorithm with three other benchmark algorithms with respect to the DMR.

In our simulation, the number of tasks ($|T_i|$) of the graph G_i is selected from a normal distribution in the range of $[1, N]$. For each task, c_j and $data_j^{res}$ are randomly chosen from $[300 \pm 10\%]$ Kilo clock cycles (KCC) and $[750 \pm 10\%]$ bits, respectively [5]. Edge E_i is generated randomly for each application by using the layer-by-layer method, which consists of three steps: determining the number of layers, distributing the tasks to the layers, and connecting the tasks from different layers [10].

We generated a network of edge nodes that are randomly deployed in an area with a size of 100×100 m, and any two edge nodes less than 35 m apart are connected to each other [5]. The values of b_{uv} were chosen from $20 \pm 10\%$ Mbps [6]. p_u is selected from $[600 \pm 10\%]$ million cycles per second (MCPS) based on Cisco fog/edge devices that perform tasks from IoT applications [4]. We then have a set of 20 input sensor data stored on random edge nodes, where two subsets of sensor data are randomly chosen for each task. The amount of input data $data_u^{in}$, is selected to be $[3200 \pm 10\%]$ bits [5].

We set the deadline δ_i , to be randomly assigned from the ranges of $[10, 20]$ ms, referring to [3], where the latency requirements for tactile Internet, factory automation, and smart grid are 7, 10, and 20 ms, respectively.

To evaluate the performance of our algorithm, the following benchmark algorithms were implemented for comparison:

- Composite DAG (cDAG): This approach creates the composite graph by making the entry nodes and the exit nodes of all the DAGs (i.e., G_i s for all a_i) immediate successors of a new entry node, and immediate ancestors of a new exit node, respectively. Then, the tasks in the cDAG are scheduled using Algorithm 1, where both input data and data between dependent tasks are considered.
- Data-Aware Earliest Deadline First (D-EDF): The multiple applications are scheduled according to EDF without considering network congestion. Then, if any

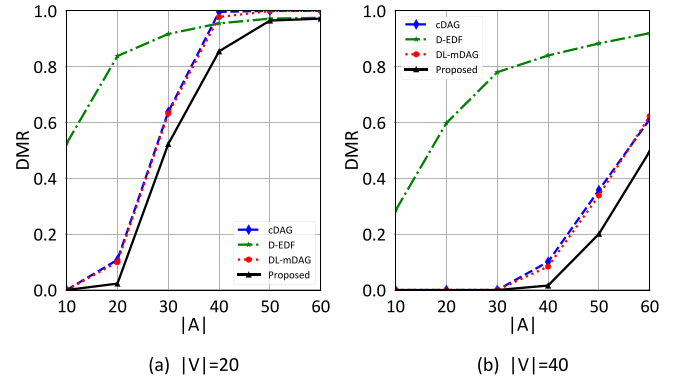


Fig. 1. DMR for different number of applications ($|A|$) for $N = 40$.

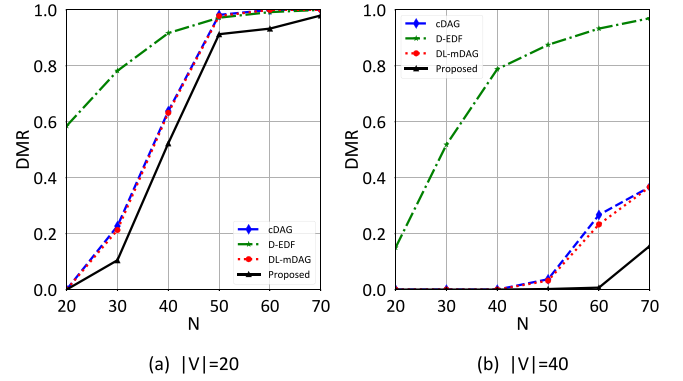


Fig. 2. DMR for different number of tasks ($|T_i|$) for $|A| = 30$.

network flow conflicts are detected, the placement of tasks and bandwidth of data flows are adjusted according to [5].

- Deadline-Aware multiple DAG (DL-mDAG): The scheduling algorithm of [6] deals with multiple DAGs; however, it is not proposed for multiple applications but for a single application. Thus, we modified the algorithm of [6] as follows: 1) We let multiple DAGs represent multiple applications. 2) While scheduling the multiple DAGs as in [6], it is checked in every stage if the RCT exceeds the deadline. If so, the tasks of the related application are scheduled until the RCT meets the deadline.

Figs. 1 (a) and (b) show the influence of the number of applications on the DMR performance of the proposed algorithm, compared to the other three algorithms, for $|V| = 20$ and 40, respectively, when $N = 40$ and the number of applications ($|A|$) varies from 10 to 60. We observe from Figs. 1 (a) and (b) that the proposed algorithm shows the smallest DMR, whereas the DMR of all four algorithms increases with an increase in $|A|$. More specifically, the proposed algorithm improves the DMR of cDAG, D-EDF, and DL-mDAG by an average of 23.5%, 41.2%, and 22.7% (48.8%, 86.9%, and 47.2%), respectively, for $|V| = 20$ ($|V| = 40$). Interestingly, D-EDF consistently performs the worst until $|A|$ increases to 30 for $|V| = 20$; however, it starts to work better than cDAG and DL-mDAG from the point where $|A| = 40$. This is because D-EDF can complete the application with the earliest deadline, whereas in cDAG and DL-mDAG, the applications compete more for the resources of edge nodes with each other as the

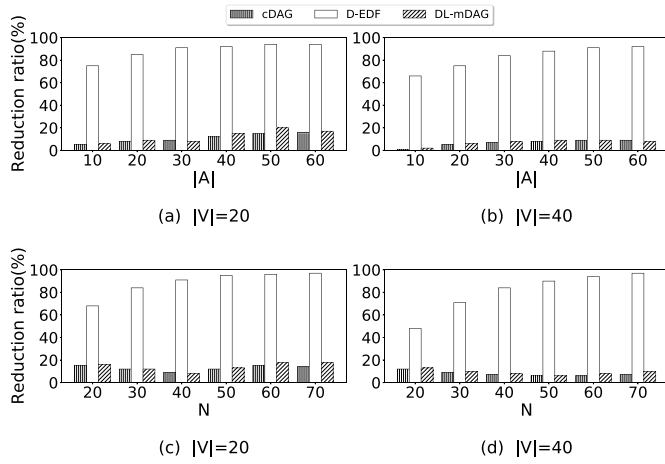


Fig. 3. The reduction ratio of the proposed algorithm in terms of the average completion time.

number of applications increases because they are more likely to be fairly scheduled than D-EDF. This phenomenon is not observed in the case of $|V| = 40$, where more computing capacity is provided for the hosted applications. Overall, DL-mDAG performs slightly better than cDAG in terms of DMR because DL-mDAG considers the applications' deadlines when scheduling the tasks, whereas cDAG does not.

Figs. 2 (a) and (b) show the DMR versus the number of tasks, $|T_i|$ for $|A| = 30$, when $|V| = 20$ and 40 , respectively. We see from this figure that the DMR of the proposed algorithm is smaller than those of the other three algorithms for the different values of N . The proposed algorithm improves the DMR of cDAG, D-EDF, and DL-mDAG by, on average, 17.5%, 40.1%, and 16.7% (83.5%, 97.2%, and 83.2%), respectively, for $|V| = 20$ ($|V| = 40$). When $|V| = 20$, the DMRs of cDAG and DL-mDAG were higher than that of D-EDF after N reached 50 for the same reason as explained in Fig. 1, i.e., the competition among applications for the edge resources becomes more intense as the applications demand more tasks to be processed within the same deadline.

Figs. 3 (a) and (b) illustrate the improvements (i.e., reduction ratio) made by the proposed algorithm over the other algorithms in terms of the average completion time when $|A|$ and N vary, respectively, for $|V| = 20$, whereas Figs. 3 (c) and (d) are for $|V| = 40$. We can see from Fig. 3 that the proposed algorithm outperforms the other algorithms in terms of the average completion time. Clearly, D-EDF has the largest average completion time because it schedules tasks and network flows separately. The proposed algorithm reduces the average completion time of cDAG and DL-mDAG by an average of 10.8% and 12.5% (6.5% and 6.9%), respectively, over the different values of $|A|$ when $|V| = 20$ ($|V| = 40$). Similarly, the average reduction ratios of the proposed algorithm are 12.7% and 14.1% (7.8% and 9.1%) for cDAG and DL-mDAG, respectively, when $|V| = 20$ ($|V| = 40$).

Table I shows the running times of our proposed algorithm and the other benchmark algorithms for different number of applications when $|V| = 40$ and $N = 40$. As shown in Table I, the running times of cDAG, DL-mDAG, and the proposed algorithms are roughly the same for the different values of

TABLE I
RUNNING TIMES (min) FOR DIFFERENT NUMBER OF APPLICATIONS ($|A|$)

$ A $	10	20	30	40	50	60
cDAG	0.8	1.9	4.4	7.7	13.8	16.6
D-EDF	2.1	5.8	16.7	25.9	42.7	81.9
DL-mDAG	0.7	1.9	4.2	7.1	12.3	15.3
Proposed	0.7	1.8	4.2	7.0	12.1	15.2

$|A|$. D-EDF has the slowest time because it requires additional running time to resolve network congestion.

V. CONCLUSION

In this letter, we have formulated a task scheduling with dependencies for the CEC system as a minimization problem of the DMR of latency-sensitive IoT applications. To solve this problem, we proposed a heuristic algorithm that determines when and where each task of multiple applications is executed, and the start time of network flows in the CEC environment, considering the applications' deadlines. Simulation results show that our proposed algorithm outperforms other benchmark methods in terms of the average completion time as well as the DMR. While we considered the average computing time in this study, one interesting future research is to investigate possible variations in the processing time at edge nodes.

REFERENCES

- [1] P. Porambage, J. Okwuibe, M. Liyanage, M. Ylianttila, and T. Taleb, "Survey on multi-access edge computing for Internet of Things realization," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 2961–2991, 4th Quart., 2018.
- [2] B. Lau, S. Marakkalage, Y. Zhou, N. Hassan, C. Yuen, and M. Zhang, "A survey of data fusion in smart city applications," *Inf. Fusion*, vol. 52, pp. 357–374, Dec. 2019.
- [3] H.A. Alameddine, S. Sharafeddine, S. Sebbah, S. Ayoubi, and C. Assi, "Dynamic task offloading and scheduling for low-latency IoT services in multi-access edge computing," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 3, pp. 668–682, Mar. 2019.
- [4] M. Al-Khafajiy, T. Baker, H. Al-Libawy, Z. Maamar, M. Aloqaily, and Y. Jararweh, "Improving fog computing performance via fog-2-fog collaboration," *Future Gener. Comput. Syst.*, vol. 100, pp. 266–280, Nov. 2019.
- [5] Y. Sahni, J. Cao, and L. Yang, "Data-aware task allocation for achieving low latency in collaborative edge computing," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 3512–3524, Apr. 2019.
- [6] Y. Sahni, J. Cao, L. Yang, and Y. Ji, "Multi-hop offloading of multiple DAG tasks in collaborative edge computing," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4893–4905, Mar. 2021.
- [7] M. Adhikari, M. Mukherjee, and S. N. Srirama, "DPTO: A deadline and priority-aware task offloading in fog computing framework leveraging multilevel feedback queueing," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 5773–5782, Jul. 2020.
- [8] L. Ale, N. Zhang, X. Fang, X. Chen, S. Wu, and L. Li, "Delay-aware and energy-efficient computation offloading in mobile edge computing using deep reinforcement learning," *IEEE Trans. Cogn. Commun. Netw.*, early access, Mar. 17, 2021, doi: [10.1109/TCCN.2021.3066619](https://doi.org/10.1109/TCCN.2021.3066619).
- [9] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.
- [10] L.-C. Canon, M. El Sayah, and P. C. Héam, "A comparison of random task graph generation methods for scheduling problems," in *Proc. Parallel Process.*, Feb. 2019, pp. 61–73.